

Patrón: Observer

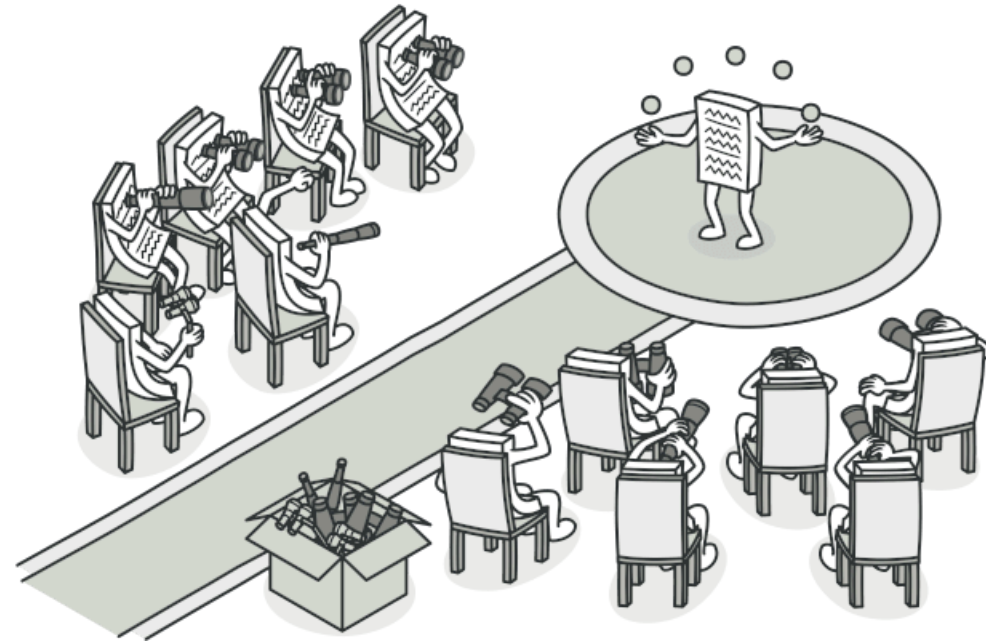
Patrón de comportamiento (POO)

Fuente principal:

<https://refactoring.guru/es/design-patterns/observer>

Propósito

Observer es un patrón de diseño de comportamiento que permite definir un **mecanismo de suscripción** para notificar a varios objetos (observadores) sobre cualquier evento que le suceda al objeto que están observando.

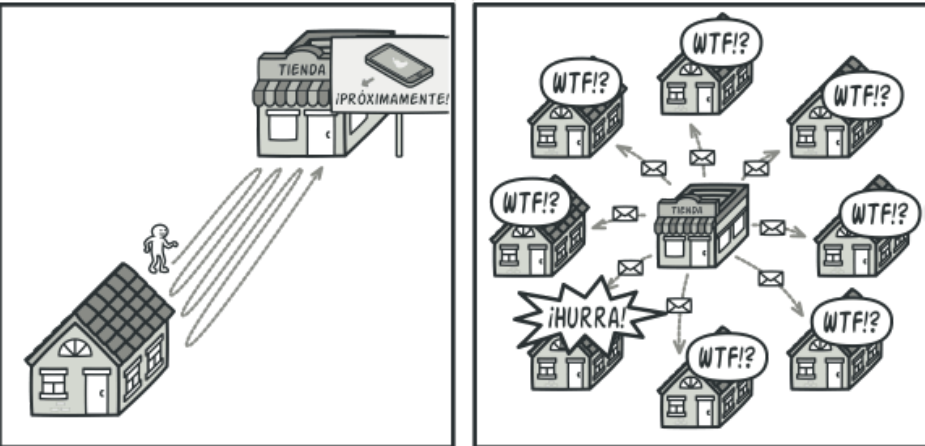


Problema

Conflicto típico:

- Un cliente quiere saber cuándo ocurre algo (p. ej. "llega el iPhone").
- Si el cliente **pregunta constantemente**, pierde tiempo y recursos.
- Si la tienda **notifica a todos**, se convierte en spam para quien no está interesado.

Objetivo: avisar **solo a los interesados**, sin acoplar al publicador con clases concretas.



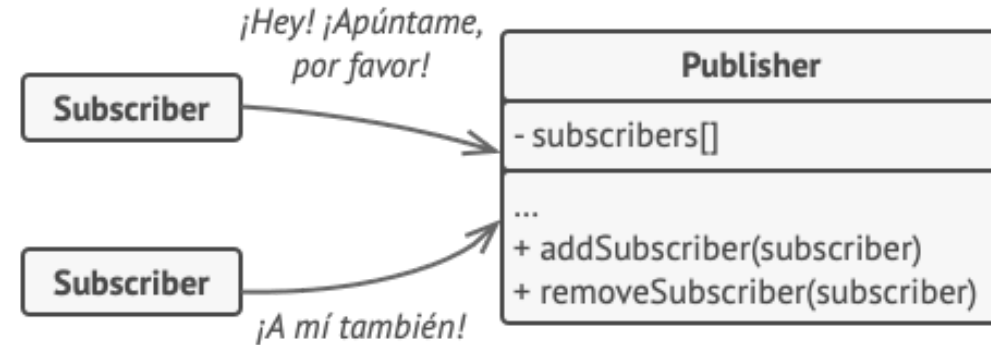
Solución

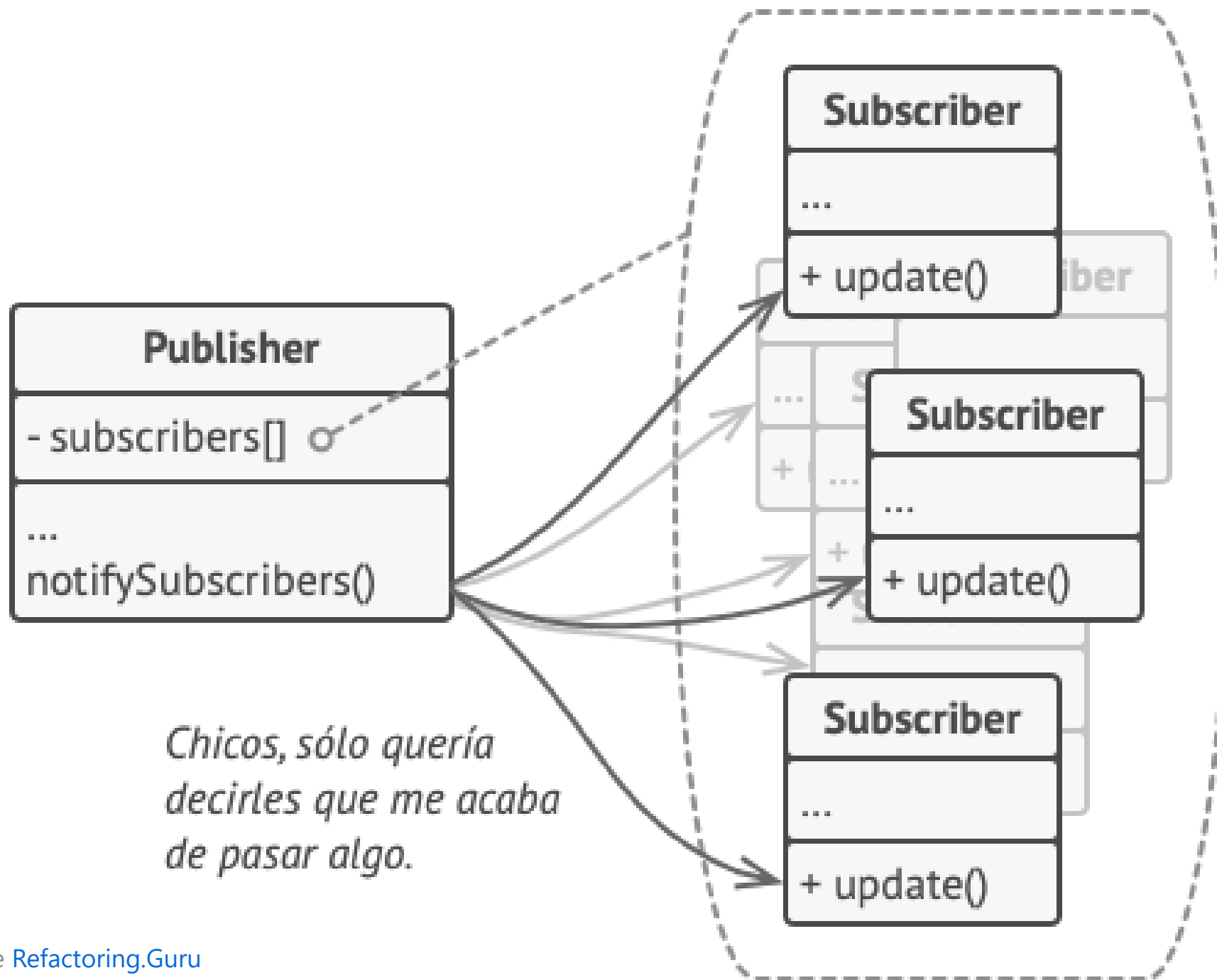
El patrón separa dos roles:

- **Publicador (Publisher / Subject):** emite eventos.
- **Suscriptores (Observers):** se registran y reaccionan.

Se añade un mecanismo de suscripción:

- Registrar / cancelar suscripción.
- Mantener una lista (a veces por tipo de evento).
- Notificar recorriendo la lista e invocando un método común (`actualizar(...)`).





Analogía en el mundo real

Suscripción a una revista/periódico:

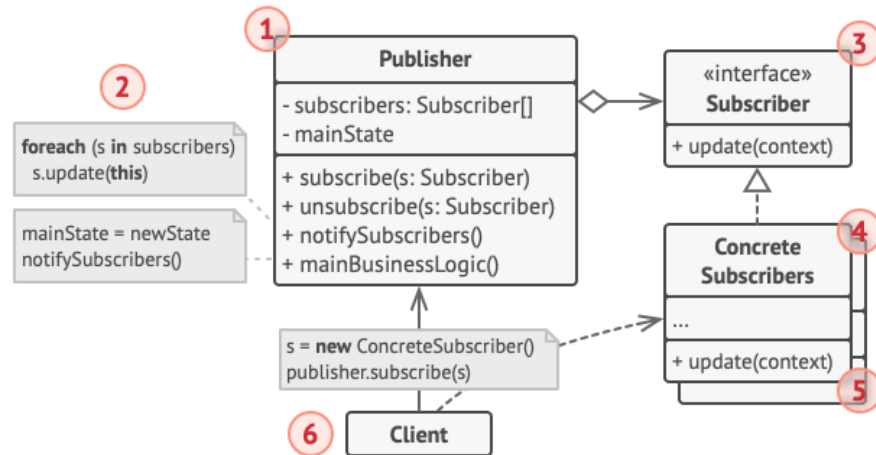
- El publicador (revista) mantiene una lista de suscriptores.
- Cuando hay un nuevo número, lo envía a quien está suscrito.
- Puedes darte de alta o de baja cuando quieras.

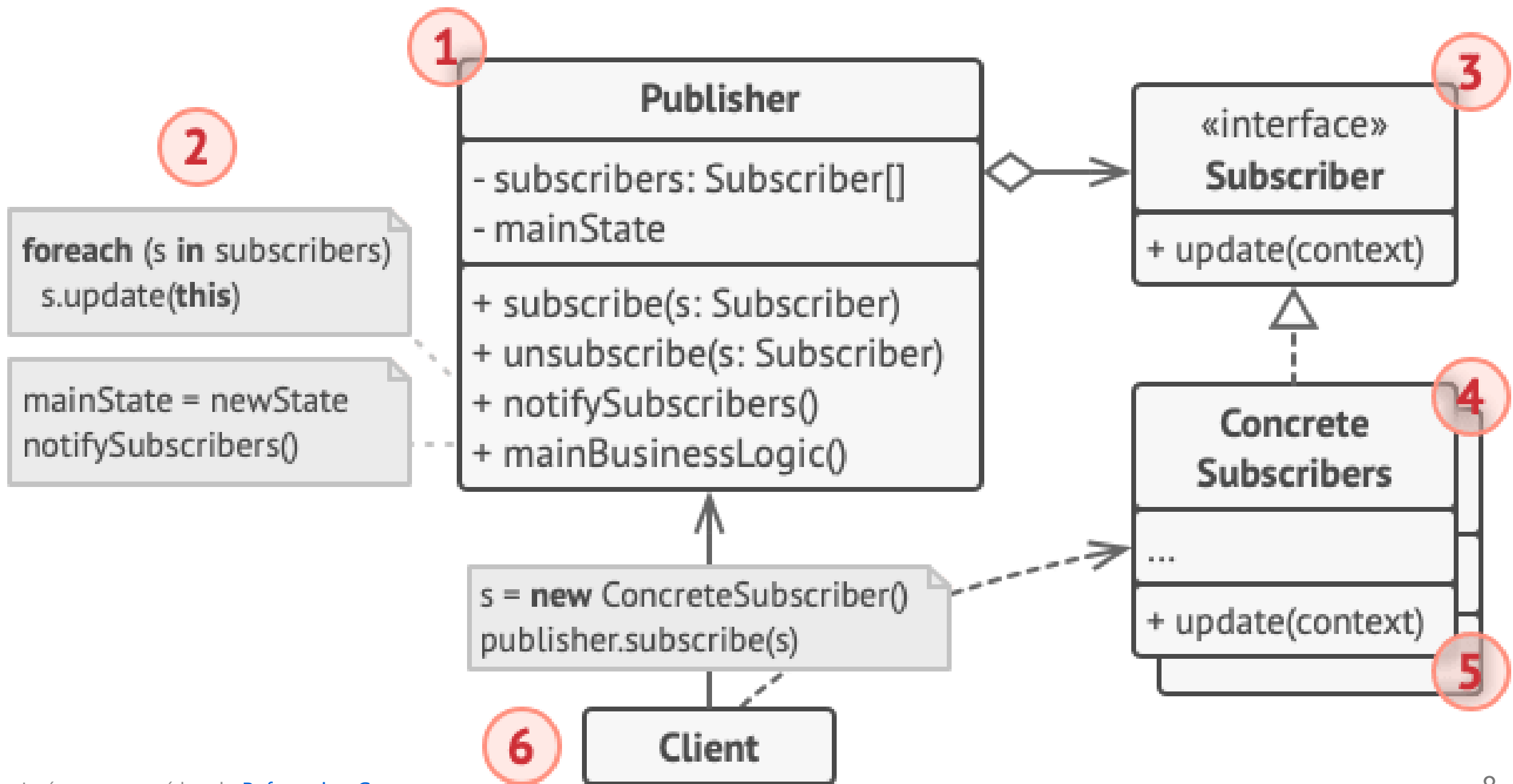


Estructura

Roles típicos:

1. **Notificador**: mantiene infraestructura de suscripción.
2. **Suscriptor**: interfaz común de notificación.
3. **Suscriptores concretos**: reaccionan ante eventos.
4. **Cliente**: crea y conecta publicador + suscriptores.

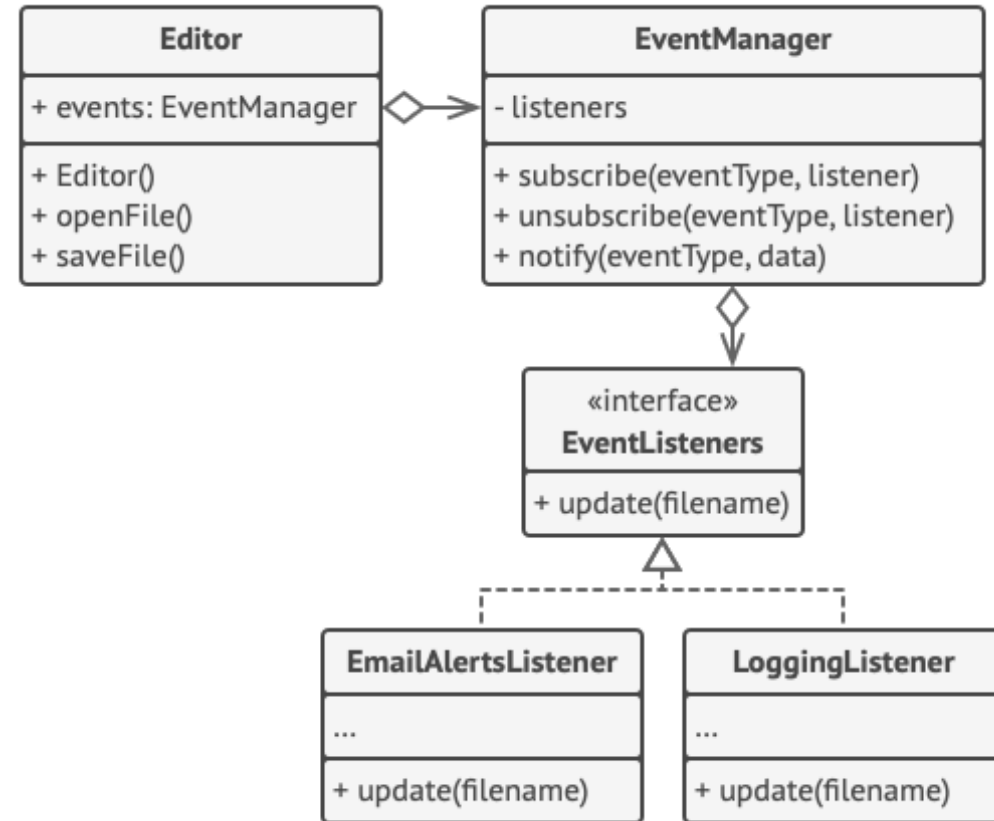


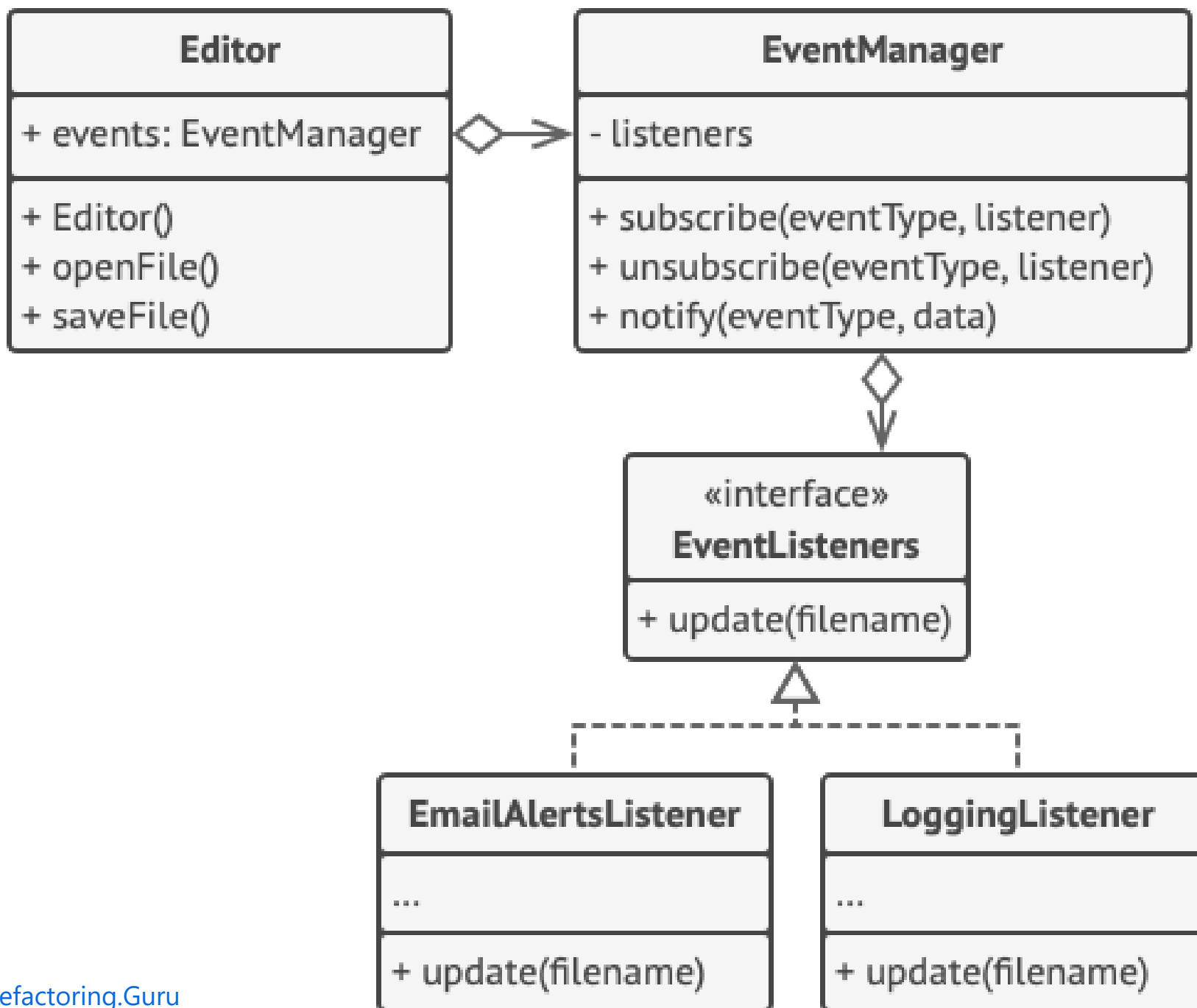


Aplicabilidad

Úsalo cuando:

- Cambios en el estado de un objeto deben disparar reacciones en otros.
- El conjunto de “interesados” es **desconocido** o **cambia dinámicamente**.
- Quieres enganchar comportamiento (listeners) durante ejecución (p. ej. GUI: clicks, cambios, guardados).





Pros y contras

Pros

- **Abierto/cerrado:** nuevos observadores sin tocar el publicador.
- Suscripciones dinámicas (alta/baja en runtime).
- Reduce acoplamiento: el publicador solo conoce una interfaz.

Contras

- El orden de notificación puede ser **indeterminado**.
- Si hay muchos observadores o eventos frecuentes, cuidado con rendimiento/ruido.

Código Java (puntos clave)

Basado en `code/es/uva/poo/observer` .

```
// 1) Interfaz del observador
public interface OyenteEvento {
    void actualizar(String tipoEvento, String datos);
}

// 2) Publicador (helper) con lista de suscriptores por evento
public class GestorEventos {
    private final Map<String, List<OyenteEvento>> oyentesPorEvento = new HashMap<>();

    public void suscribir(String tipoEvento, OyenteEvento oyente) {
        oyentesPorEvento.computeIfAbsent(tipoEvento, k -> new ArrayList<>()).add(oyente);
    }

    public void notificar(String tipoEvento, String datos) {
        List<OyenteEvento> oyentes = oyentesPorEvento.get(tipoEvento);
        if (oyentes == null) return;
        for (OyenteEvento oyente : new ArrayList<>(oyentes)) {
            oyente.actualizar(tipoEvento, datos);
        }
    }
}
```

Código Java (conexión en runtime)

```
public class Demo {  
    public static void main(String[] args) {  
        Editor editor = new Editor();  
  
        OyenteEvento email = new OyenteNotificacionEmail("profesor@universidad.es");  
        OyenteEvento log = new OyenteRegistro("eventos.log");  
  
        editor.eventos.suscribir("abrir", email);  
        editor.eventos.suscribir("guardar", email);  
        editor.eventos.suscribir("abrir", log);  
        editor.eventos.suscribir("guardar", log);  
  
        editor.abrirArchivo("ejemplo.txt");  
        editor.guardarArchivo();  
    }  
}
```